



CÍRCULO POLICIAL DE FLORES

Sistema de Gestión de Reservas

MANUAL Técnico

Integración con Azure Document Intelligence

Análisis automático de facturas mediante OCR para el módulo de Finanzas

Módulo / Alcance	Finanzas — Análisis de Facturas (Azure Document Intelligence)
Dirigido a	Equipo de desarrollo y mantenimiento de la aplicación
Versión	1.0.0
Fecha	27/07/2026
Estado	Aprobado



INFORMACIÓN DEL DOCUMENTO

Campo	Detalle
Título del documento	Manual Técnico — Integración con Azure Document Intelligence
Módulo / Sistema	Finanzas (Análisis automático de facturas)
Tipo de manual	Técnico
Público objetivo	Equipo de desarrollo del proyecto CIPOLFLO
Aplicación / versión del sistema	cipolflo-server — SDK azure-ai-documentintelligence 1.0.0-beta.4 (API 2024-07-31-preview)
Última actualización	27/07/2026
Estado	Aprobado

HISTORIAL DE VERSIONES

Versión	Fecha	Autor	Descripción de cambios
0.1	27/07/2026	Equipo CIPOLFLO	Versión inicial del manual, a partir de la implementación de la integración con Azure Document Intelligence en Sprint 6.

DOCUMENTOS RELACIONADOS

- Manual de usuario - Módulo de finanzas



Índice

Índice	3
1. Introducción.....	4
1.1 Objetivo del documento.....	4
1.2 Alcance y público destinatario.....	4
1.3 Requisitos previos.....	4
2. Conceptos clave	4
3. Configuración del recurso en Azure.....	5
3.1 Creación del recurso en Azure Portal	5
3.2 Obtención del endpoint y la key.....	5
4. Integración en el backend (Spring Boot)	6
4.1 Dependencia en build.gradle.kts	6
4.2 Configuración de credenciales	6
4.3 Clase de propiedades y bean del cliente	6
4.4 Entidad, repositorio y persistencia.....	6
4.5 Servicio DocumentoAzureService.....	7
4.6 Controller REST y seguridad	7
4.8 Validaciones recomendadas	7
5. Integración en el frontend (Angular)	8
5.1 Formulario de carga.....	8
5.2 Validaciones del lado del cliente	8
5.3 Proxy de desarrollo.....	8
6. Campos que devuelve el modelo prebuilt-invoice	8
7. Límites y cuotas	9
8. Variables de entorno para CI/CD y Docker Compose	9
9. Preguntas frecuentes / Solución de problemas.....	9
10. Glosario.....	10

1. Introducción

1.1 Objetivo del documento

Este manual describe la integración del sistema CIPOLFLO con el servicio Azure Document Intelligence para el análisis automático de facturas mediante el modelo precompilado prebuilt-invoice. Al finalizar su lectura, el desarrollador podrá configurar el recurso en Azure, implementar el flujo completo en el backend (Spring Boot) y en el frontend (Angular), e interpretar los campos extraídos por el servicio.

1.2 Alcance y público destinatario

Dirigido al equipo de desarrollo del proyecto CIPOLFLO, con conocimientos de Java, Spring Boot y Angular. Cubre la creación del recurso en Azure, la integración backend y frontend, los campos devueltos por el modelo, los límites del plan gratuito y la resolución de errores comunes. Quedan fuera del alcance el entrenamiento de modelos personalizados de Azure y el módulo de importación de Excel, que se documentan por separado.

1.3 Requisitos previos

Antes de continuar, es necesario contar con lo siguiente:

- Cuenta de Microsoft con acceso a una suscripción activa de Azure
- Acceso al repositorio del proyecto (cipolflo-server) con Java 21 y Gradle configurados
- Angular CLI y acceso al proyecto frontend
- Permisos para crear recursos dentro del grupo de recursos del proyecto

2. Conceptos clave

Azure Document Intelligence es un servicio cloud de Microsoft que recibe un archivo (PDF o imagen) y devuelve un JSON con el texto y los campos extraídos mediante OCR e inteligencia artificial. En CIPOLFLO se utiliza el modelo precompilado prebuilt-invoice, entrenado por Microsoft para reconocer facturas.

2.1 Flujo de integración

Componente	Responsabilidad
Angular (frontend)	El usuario selecciona el archivo y lo envía con FormData por HTTP POST
Spring Boot Controller	Recibe el MultipartFile y lo delega al servicio
DocumentoAzureService	Convierte el archivo, llama a Azure y guarda el JSON resultado
PostgreSQL	Almacena el resultado estructurado para consultas posteriores

Azure Document Intelligence	Procesa el documento y devuelve el JSON con los campos extraídos
-----------------------------	--

⚠ Importante

El SDK GA (versión 1.0.0) de Azure para Java apunta a la API 2024-11-30 (v4.0 GA), pero aún no está disponible como artefacto estable en todas las configuraciones de Maven.

Por eso el proyecto utiliza la versión 1.0.0-beta.4, que apunta a la API 2024-07-31-preview y ofrece exactamente la misma interfaz para prebuilt-invoice.

Cuando la versión GA esté disponible, la actualización es directa: solo cambia el número de versión en build.gradle.kts.

3. Configuración del recurso en Azure

Este paso se realiza una única vez por entorno. Una vez creado el recurso se obtienen el endpoint y la key que utilizará toda la integración.

3.1 Creación del recurso en Azure Portal

1. Ingresar a portal.azure.com con la cuenta de Microsoft del proyecto.
2. Buscar "Document Intelligence" y seleccionar el servicio.
3. Hacer clic en "Crear" y completar el formulario (suscripción, grupo de recursos, región East US, nombre descriptivo, plan de tarifa F0 para desarrollo).
4. Hacer clic en "Revisar y crear" y esperar la validación automática.
5. Al aparecer "Validación superada", hacer clic en "Crear" y esperar a que finalice la implementación.

3.2 Obtención del endpoint y la key

Dentro del recurso, en "Claves y punto de conexión", se obtienen dos datos críticos: el endpoint (URL del recurso) y la Key 1 (cadena hexadecimal de 32 caracteres).

⚠ Importante

Las credenciales nunca deben subirse al repositorio.

Se utilizan exclusivamente como variables de entorno, nunca como valores fijos en el código.

Si la key se sube por error a GitHub, Azure la invalida automáticamente y debe regenerarse.

3.3 Prueba en Document Intelligence Studio

Antes de escribir código, se recomienda validar el modelo directamente en documentintelligence.ai.azure.com/studio: seleccionar el recurso creado, buscar el modelo "Invoice" bajo "Prebuilt models" y probarlo con una factura real en PDF, JPG o PNG, revisando que campos como VendorName, InvoiceDate, InvoiceTotal e Items se extraigan correctamente.

i Nota

Si el Studio extrae correctamente los campos necesarios, se utiliza prebuilt-invoice sin configuración adicional del lado de Azure.

Si existen campos propios que el modelo genérico no reconoce, se podría evaluar entrenar un modelo personalizado en un sprint futuro.

4. Integración en el backend (Spring Boot)

4.1 Dependencia en build.gradle.kts

Se agregan las siguientes dependencias dentro del bloque dependencies { } de cipolflo-server:

```
implementation("com.azure:azure-ai-documentintelligence:1.0.0-beta.4")
implementation("com.azure:azure-core:1.53.0")
```

azure-ai-documentintelligence aporta DocumentIntelligenceClient y las clases del dominio de Azure DI; azure-core aporta BinaryData, AzureKeyCredential y SyncPoller. Luego de agregarlas se debe sincronizar el proyecto (./gradlew dependencies o recargar Gradle desde IntelliJ) y regenerar el verification-metadata.xml correspondiente.

4.2 Configuración de credenciales

En application.yml se agrega:

```
azure:
  document-intelligence:
    endpoint: ${AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT}
    key: ${AZURE_DOCUMENT_INTELLIGENCE_KEY}
```

Las variables se setean localmente por entorno (PowerShell, configuración de Run en IntelliJ, o archivo .env vía spring-dotenv). Si una variable no está definida, Spring Boot falla al iniciar indicando cuál falta.

4.3 Clase de propiedades y bean del cliente

AzureDocumentIntelligenceProperties (en com.cipolflo.server.shared.config) es un record anotado con @ConfigurationProperties(prefix = "azure.document-intelligence") que expone endpoint y key. AzureDocumentIntelligenceConfig declara el bean singleton DocumentIntelligenceClient mediante DocumentIntelligenceClientBuilder, configurando endpoint y credencial (AzureKeyCredential).

i Nota

Si la key es incorrecta o expiró, la llamada a Azure devuelve HttpResponseException 401 Unauthorized. Verificar que las variables de entorno estén correctamente seteadas antes de levantar la aplicación.

4.4 Entidad, repositorio y persistencia

DocumentoAnalizado (com.cipolflo.server.documentos.model) es la entidad JPA que registra cada análisis: nombreArchivo, tipoContenido, modeloUsado, fechaAnálisis y resultadoJson (columna TEXT con el JSON

completo devuelto por Azure, incluyendo campos, niveles de confianza y coordenadas).

DocumentoAnalizadoRepository extiende JpaRepository y agrega búsquedas por modelo y por nombre de archivo.

4.5 Servicio DocumentoAzureService

Es el componente central de la integración. Convierte el MultipartFile recibido en un AnalyzeDocumentRequest, invoca client.beginAnalyzeDocument(...) con el modelo prebuilt-invoice y obtiene el resultado mediante un SyncPoller, que abstrae el procesamiento asíncronico de Azure: internamente realiza polling hasta que el análisis está listo (en promedio entre 1 y 5 segundos para una factura simple). El resultado se serializa a JSON y se persiste junto con los metadatos del archivo.

⚠ Importante

En producción se recomienda fijar un timeout explícito, por ejemplo poller.waitForCompletion(Duration.ofSeconds(30)), ya que un archivo grande o Azure bajo carga pueden demorar la respuesta.

4.6 Controller REST y seguridad

DocumentoAzureController expone POST /api/documentos/analizar-factura, que recibe el archivo como multipart/form-data (campo file) y delega en el servicio. Al estar el proyecto protegido con Auth0, el endpoint debe quedar cubierto por la regla general de autenticación (o explícitamente con .requestMatchers("/api/documentos/**").authenticated() en SecurityConfig).

4.7 Configuración de multipart

Spring Boot limita por defecto la subida de archivos a 1 MB. Se ajusta en application.yml:

```
spring:
  servlet:
    multipart:
      enabled: true
      max-file-size: 4MB
      max-request-size: 5MB
```

ℹ Nota

El límite de 4 MB corresponde al máximo admitido por el plan gratuito (F0) de Azure Document Intelligence; no tiene sentido permitir archivos más grandes si Azure los va a rechazar. Si el proyecto migra al plan S0, este valor puede aumentarse.

4.8 Validaciones recomendadas

Antes de invocar a Azure, el servicio valida que el archivo no esté vacío, que no supere los 4 MB y que su content-type pertenezca a los formatos soportados (PDF, JPEG, PNG, BMP, TIFF, HEIF), lanzando IllegalArgumentException con un mensaje claro en caso contrario. Un GlobalExceptionHandler ya existente en el proyecto centraliza el manejo de estas excepciones y de los errores internos, devolviendo respuestas homogéneas al frontend.

5. Integración en el frontend (Angular)

La integración del lado de Angular consiste en un input de tipo archivo y un botón que envía el contenido como FormData al endpoint del backend.

5.1 Formulario de carga

El componente permite seleccionar un archivo (accept=".pdf,.jpg,.jpeg,.png,.bmp,.tiff,.heif"), muestra el estado "Analizando..." mientras espera la respuesta y presenta el resultado o el mensaje de error devuelto por el backend.

5.2 Validaciones del lado del cliente

Antes de enviar el archivo se valida en el propio componente que el tipo MIME esté entre los permitidos y que el tamaño no supere los 4 MB, evitando así solicitudes innecesarias al backend.

5.3 Proxy de desarrollo

Como Angular corre en el puerto 4200 y Spring Boot en el 8080, se configura un proxy de desarrollo (proxy.conf.json) que redirige las solicitudes a /api hacia <http://localhost:8080>, evitando problemas de CORS en el entorno local.

6. Campos que devuelve el modelo prebuilt-invoice

No todos los campos estarán presentes en todas las facturas; su disponibilidad depende del formato y contenido del documento analizado.

Campo	Tipo	Descripción
VendorName	STRING	Nombre del proveedor
CustomerName	STRING	Nombre del cliente
InvoiceId	STRING	Número de factura
InvoiceDate	DATE	Fecha de emisión
DueDate	DATE	Fecha de vencimiento
InvoiceTotal	CURRENCY	Total de la factura
SubTotal	CURRENCY	Subtotal antes de impuestos
TotalTax	CURRENCY	Total de impuestos
AmountDue	CURRENCY	Monto pendiente de pago
PaymentTerm	STRING	Condiciones de pago
Items	ARRAY	Líneas de detalle: descripción, cantidad, precio

i Nota

Cada campo expone un nivel de confianza (`getConfidence()`, entre 0.0 y 1.0). Un valor mayor a 0.9 indica alta confianza; por debajo de 0.7 conviene revisar el campo manualmente.

En producción puede resultar útil mostrar una alerta visual cuando la confianza de un campo es baja.

7. Límites y cuotas

Cuota / Límite	Plan F0 (Gratuito)	Plan S0 (Estándar)
Tamaño máximo del documento	4 MB	500 MB
Páginas máximas por análisis	2 páginas	2.000 páginas
Transacciones por segundo	1 TPS	15 TPS (ajustable)
Formatos soportados	PDF, JPEG/JPG, PNG, BMP, TIFF, HEIF	Ídem
PDF con contraseña	No soportado	No soportado

⚠ Importante

En el plan gratuito F0, si se sube un PDF de varias páginas, Azure procesa únicamente las primeras dos. No es un error: es el límite documentado del plan. Para una factura estándar de una página no representa un problema; solo afecta documentos multipágina.

8. Variables de entorno para CI/CD y Docker Compose

En `docker-compose.yml` se agregan `AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT` y `AZURE_DOCUMENT_INTELLIGENCE_KEY` al servicio del backend, tomadas del archivo `.env` local (nunca versionado). En GitHub Actions, ambas variables se configuran como secrets del repositorio y se referencian en el workflow de CI.

i Nota

Los tests unitarios de `DocumentoAzureService` deben mockear `DocumentIntelligenceClient` con Mockito; no deben llamar a Azure real, para evitar flakiness y no consumir cuota del servicio.

9. Preguntas frecuentes / Solución de problemas

Error	Causa probable	Solución
-------	----------------	----------



401 Unauthorized	Key incorrecta o variables de entorno no seteadas	Verificar AZURE_DOCUMENT_INTELLIGENCE_KEY y reiniciar la app
404 Not Found en el endpoint	URL mal formada o recurso eliminado	Copiar el endpoint exacto desde Azure Portal (debe terminar en /)
HttpResponseException 400	Formato no soportado o archivo corrupto	Verificar el content-type; probar con otro archivo
FileSizeLimitExceededException	El archivo supera los límites de Spring Boot	Aumentar max-file-size en application.yml
No se puede inyectar DocumentIntelligenceClient	Falta @EnableConfigurationProperties o falta @Bean	Revisar AzureDocumentIntelligenceConfig
NullPointerException en getDocuments()	Resultado vacío o documento no reconocido	Validar result.getDocuments() != null antes de iterar
CORS error desde Angular	Frontend y backend en orígenes distintos sin proxy	Configurar proxy.conf.json o @CrossOrigin

10. Glosario

Término	Definición
OCR	Reconocimiento óptico de caracteres: extracción de texto a partir de imágenes o documentos escaneados
SyncPoller	Componente del SDK de Azure que abstrae operaciones asíncronas de larga duración mediante polling
Modelo prebuilt-invoice	Modelo de IA precompilado de Azure, entrenado para reconocer campos típicos de facturas
Confidence	Nivel de confianza (0.0 a 1.0) que Azure asigna a cada campo extraído